

Plano de Desenvolvimento Atualizado - Trabalho Prático de Engenharia de Dados

Repositório: <<https://github.com/thejosephantony/ProjetoFinal-EngenhariaDeDados>>

Data da atualização: 28/06/2026

1. Objetivo do projeto

O projeto tem como objetivo desenvolver uma solução prática de Engenharia de Dados dividida em três frentes principais: manipulação de dados em banco relacional, manipulação de dados em banco NoSQL e integração de dados para construção de um modelo analítico.

A primeira frente consiste em desenvolver uma aplicação que se comunica com um SGBD PostgreSQL e executa operações de CRUD sobre tabelas do esquema relacional trabalhado em aula. As tabelas centrais da Parte 1 são **usuario**, **estudante**, **curso** e **vinculo**, todas pertencentes ao schema **universidade**.

2. Organização geral do projeto

```
ProjetoFinal-EngenhariaDeDados/  
├── CRUD Relacional - Parte1/  
├── CRUD NoSQL - Parte2/  
├── Integração de dados - Parte3/  
├── dis-csv-discentes-de-graduacao-de-2025_1.csv  
├── universidade-dump-engdados.sql  
├── README.md  
└── Trabalho Prático de Engenharia de Dados 2026.1.pdf
```

3. Arquitetura da Parte 1

A Parte 1 foi implementada como uma API REST usando Java com Spring Boot. A aplicação segue uma divisão em camadas:

Controller -> Service -> Repository -> Banco PostgreSQL

Camada Controller

Responsável por expor os endpoints HTTP da API.

Pacote:

br.ufs.engenhariadados.controller

Principais controllers:

```
UsuarioController  
EstudanteController  
CursoController  
VinculoController
```

Camada DTO

Responsável por separar os dados recebidos e retornados pela API das entidades persistidas no banco.

Pacote:

br.ufs.engenhariadados.dto

Principais DTOs:

UsuarioRequestDTO / UsuarioResponseDTO
EstudanteRequestDTO / EstudanteResponseDTO
CursoRequestDTO / CursoResponseDTO
VinculoRequestDTO / VinculoResponseDTO

Camada Service

Responsável pelas regras de negócio, validações, busca de relacionamentos e tratamento de operações de CRUD.

Pacote:

br.ufs.engenhariadados.service

Principais services:

UsuarioService
EstudanteService
CursoService
VinculoService

Camada Repository

Responsável pela comunicação com o banco usando Spring Data JPA.

Pacote:

br.ufs.engenhariadados.repository

Principais repositories:

UsuarioRepository
EstudanteRepository
CursoRepository
VinculoRepository

Camada Model

Responsável pelo mapeamento JPA das tabelas oficiais do schema **universidade**.

Pacote:

br.ufs.engenhariadados.model

Principais entidades:

Usuario
Estudante
Curso
Vinculo

4. Banco de dados da Parte 1

A aplicação usa PostgreSQL e o schema **universidade**, criado a partir do arquivo:

universidade-dump-engdados.sql

Banco usado localmente:

engenharia_dados

Schema:

universidade

Tabelas principais do CRUD:

universidade.usuario
universidade.estudante
universidade.curso
universidade.vinculo

5. Mapeamento das tabelas principais

5.1 Usuario

A tabela **usuario** representa usuários gerais do sistema, incluindo professores e estudantes.

Campos principais:

cpf
data_nascimento
email[]
login
nome
senha
telefone[]

Chave primária:

cpf

Observação: a senha não é retornada nas respostas da API.

5.2 Estudante

A tabela **estudante** representa estudantes associados a usuários.

Campos principais:

mat_estudante
cpf
mc
ano_ingresso

Chave primária:

mat_estudante

Relacionamento:

estudante.cpf -> usuario.cpf

5.3 Curso

A tabela **curso** representa cursos da universidade.

Campos principais:

idcurso
nome
grau
turno
campus
nivel

Chave primária:

idcurso

Tipos enumerados usados:

tipo_grau: Bacharelado, Licenciatura Plena
tipo_turno: Matutino, Vespertino, Noturno, Turno Indefinido
tipo_nivel: Graduação, Mestrado, Doutorado, Lato

5.4 Vinculo

A tabela **vinculo** representa o vínculo entre um estudante e um curso.

Campos principais:

idvinculo
mat_estudante
curso
data_entrada
status
data_saida

Chave primária:

idvinculo

Relacionamentos:

vinculo.mat_estudante -> estudante.mat_estudante
vinculo.curso -> curso.idcurso

Status válidos:

Ativo
Cancelada
Formando
Graduado

6. Status da Parte 1

A Parte 1 está concluída no ambiente local.

Entidade	Create	Read	Update	Delete	Status
Usuario	OK	OK	OK	OK	Concluído localmente
Estudante	OK	OK	OK	OK	Concluído localmente
Curso	OK	OK	OK	OK	Concluído localmente
Vinculo	OK	OK	OK	OK	Concluído localmente

7. Testes realizados na Parte 1

Foram realizados testes com **Invoke-RestMethod** no PowerShell.

7.1 Criação de usuário

Usuário criado com sucesso:

```
cpf = 3333333301  
nome = Usuario Teste Java  
login = usuario.teste.java
```

7.2 Criação de estudante

Estudante criado com sucesso:

```
matricula = 3330001  
cpf = 3333333301  
mc = 80  
anoIngresso = 2025
```

7.3 Criação de curso

Curso criado com sucesso após correção da sequence:

```
idCurso = 7  
nome = Curso Teste Java  
turno = Noturno  
campus = Campus Teste
```

7.4 Criação de vínculo

Vínculo criado com sucesso:

```
idVinculo = 15  
matriculaEstudante = 3330001  
cursoId = 7  
status = Ativo
```

7.5 Atualização e remoção

Foram testadas operações de atualização e remoção respeitando a ordem das chaves estrangeiras:

1. Remover vínculo
2. Remover estudante
3. Remover curso
4. Remover usuário

8. Problemas encontrados e correções realizadas

8.1 Modelo inicial simplificado

Inicialmente foi criado um modelo simplificado baseado no CSV de discentes. Depois foi identificado que a Parte 1 deveria usar o schema relacional oficial trabalhado em aula. O código foi refatorado para usar o dump **universidade-dump-engdados.sql**.

8.2 Encoding dos caracteres

Durante a importação do dump, nomes como **Ciência da Computação, São Cristóvão e Graduação** apareceram com caracteres quebrados. A correção foi importar o dump com UTF-8:

```
chcp 65001
$env:PGCLIENTENCODING="UTF8"
```

8.3 Sequences do PostgreSQL

Após a importação do dump, as sequences das tabelas **curso** e **vinculo** precisaram ser atualizadas para evitar colisão de chave primária.

Comandos usados:

```
SELECT setval(
    'universidade.curso_idcurso_seq',
    (SELECT MAX(idcurso) FROM universidade.curso)
);

SELECT setval(
    'universidade.vinculo_idvinculo_seq',
    (SELECT MAX(idvinculo) FROM universidade.vinculo)
);
```

9. Próximo passo da Parte 1: AWS RDS

Embora a Parte 1 esteja funcionando localmente, o enunciado exige PostgreSQL hospedado na AWS. Portanto, a próxima etapa é migrar a base para o Amazon RDS.

Tarefas da AWS

- Criar uma instância PostgreSQL no Amazon RDS.
- Configurar usuário, senha, porta e banco.
- Ajustar security group para permitir acesso apenas dos IPs necessários.
- Importar o dump `universidade-dump-engdados.sql` no banco remoto.
- Corrigir sequences no banco remoto.
- Ajustar as variáveis de ambiente do Spring Boot.
- Testar todos os endpoints com o banco remoto.
- Registrar evidências para apresentação.

Variáveis recomendadas

```
SPRING_DATASOURCE_URL
SPRING_DATASOURCE_USERNAME
SPRING_DATASOURCE_PASSWORD
```

Exemplo de URL:

```
jdbc:postgresql://ENDPOINT_RDS:5432/engenharia_dados?currentSchema=universidade
```

10. Plano da Parte 2 - CRUD NoSQL

A Parte 2 ainda está pendente. A proposta é implementar um CRUD NoSQL separado, mantendo a organização do repositório.

Tarefas previstas

- Definir o banco NoSQL.
- Definir o domínio a ser persistido.
- Criar estrutura da aplicação.
- Implementar operações de criação, leitura, atualização e remoção.
- Criar exemplos de documentos.
- Testar endpoints.
- Documentar decisões de modelagem.

Possível abordagem

Usar MongoDB para modelar documentos relacionados ao contexto acadêmico, por exemplo:

```
historico_academico
atividades_academicas
perfil_estudante
logs_acesso
```

A escolha final deve seguir o enunciado da Parte 2 e as orientações do professor.

11. Plano da Parte 3 - Integração de Dados e Esquema Estrela

A Parte 3 envolve construção de modelo analítico e ETL.

Objetivo

Construir um esquema estrela para análise de turmas de graduação, integrando dados do banco relacional e arquivos CSV externos.

Ferramenta prevista

Apache Hop

Fontes previstas

- Banco relacional da Parte 1.
- CSV de Unidades Acadêmicas.
- CSV de Componentes Curriculares.
- CSV de Docentes.
- CSV de Turmas.
- Turmas entre 2019 e 2025.

Modelo analítico previsto

Dimensões candidatas:

```
Dim_Tempo
Dim_Curso
Dim_Departamento
Dim_Disciplina
Dim_Professor
Dim_Campus
Dim_Semestre
```

Fato candidata:

Fato_Turma

Métricas candidatas:

número de matriculados
média de notas
quantidade de aprovados
quantidade de reprovados

12. Cronograma sugerido

Fase	Atividade	Status
1	Preparação do repositório	Concluído
2	Configuração do PostgreSQL local	Concluído
3	Importação do dump oficial	Concluído
4	Refatoração para schema `universidade`	Concluído
5	CRUD relacional local	Concluído
6	Documentação da Parte 1 local	Em andamento
7	PostgreSQL na AWS RDS	Pendente
8	Testes da Parte 1 na AWS	Pendente
9	CRUD NoSQL	Pendente
10	Integração de dados com Apache Hop	Pendente
11	Esquema estrela	Pendente
12	Relatório final e apresentação	Pendente

13. Critérios de qualidade

A entrega deve demonstrar:

- Uso correto do schema relacional oficial.
- CRUD completo nas quatro tabelas exigidas.
- Comunicação real com PostgreSQL.
- Banco hospedado na AWS para a entrega da Parte 1.
- Código organizado em camadas.
- Tratamento de erros.
- Validações mínimas de entrada.
- Evidências de teste.
- README atualizado.
- Plano de desenvolvimento coerente com o andamento do projeto.

14. Comandos Git recomendados

```
cd "C:\Users\Joseph\Downloads\ProjetoFinal-EngenhariaDeDados"  
git status  
git add -A  
git commit -m "Atualiza documentação da Parte 1 relacional"
```


git push

15. Situação final desta atualização

Nesta atualização, o projeto passa a registrar oficialmente que:

- O dump oficial do schema universidade foi importado.
- O backend Spring Boot foi refatorado para o schema real.
- O CRUD local de usuario, estudante, curso e vinculo foi validado.
- A próxima pendência crítica é migrar o PostgreSQL para AWS RDS.